a doctor, patient, or admin, shares the same core schema, with a role field that drives what they can see and do on the platform. We also included fields like:

- personalInfo (name, contact, etc)
- medicalSpecialization (for doctors)
- status (for account approval)
- permissions (derived from the role)

Appointment model: This model links patients and doctors together and stores scheduling details. It includes:

- patientId, doctorId
- schedule object with date and time
- status flags (requested, approved, completed)
- timestamps for tracking the appointment lifecycle

Group model: Support groups are central to the platform's engagement model. Each group has:

- unique groupId
- name, description, category
- list of members
- admin/moderator role assignment

We modelled it so that group membership and moderation privileges could scale independently.

Resource model*:* When doctors upload content (PDFs, links, etc), it gets stored in the resource model, which holds:

- title, author, fileType
- URL to access the file
- tags and categories for filtering
- upload date and permission scope (public/private)

Design considerations

Here are a few things we learned and applied while modelling our data:

Embedded vs referenced*:* We embedded small fields like user roles inside the user document but used references (_id) for things like appointments and groups to keep the models lean and flexible.

Timestamps everywhere*:* For almost every model, we added createdAt and updatedAt. It came in handy during dashboard analytics and debugging.

Indexing*:* We indexed fields like email, status, and doctorId to keep queries fast, especially when the data started to grow.

Sensitive field protection*:* Passwords, tokens, and sensitive metadata were excluded from responses using Mongoose schema options like *select: false*.

Data modelling was one of those behind-the-scenes things that made a big difference later. Because our models were well-structured from the beginning, adding new features like audit trails or file tagging didn't require rewrites. If we had gone with a more rigid schema, many of the iterations we did during testing would've been painful to implement.

Security and authentication

With SereneCare dealing directly with user data (some of it potentially sensitive), it was clear from the start that security couldn't be an afterthought. We tried to bake in protection mechanisms across the full stack, keeping things lightweight without compromising on best practices.

JWT-based authentication: For login and session management, we used JSON Web Tokens (JWT). Every time a user logs in successfully, they receive a signed token that contains their user ID and role. This token is then sent with every request to protected routes. Why JWT?

- It allowed us to keep the backend stateless, which made scaling easier.
- We could encode role data directly into the token, avoiding extra DB lookups.
- Tokens expire after a set duration, so sessions are automatically invalidated over time.

We used middleware in Express to intercept and validate these tokens before passing requests along to protected endpoints.

Role-based access control (RBAC): One of the most effective security patterns we implemented was RBAC. Each user in the system, whether patient, doctor, or admin, has permissions tied to their role, and this controls what APIs they can access and what actions they're allowed to take.

Here's how we enforced it:

- The user's role was extracted from the JWT.
- Each route group had role-checking middleware to gate access.
- On the frontend, we also used this role information to render only relevant UI components.

This setup ensured that, for instance, a patient could never accidentally hit a doctor-only route or access group moderation features.

Password handling with Bcrypt: For user credentials, we took a straightforward approach: no plain-text passwords, ever.

- On signup, we used *bcrypt.hash()* to securely hash the password before storing it in the database.
- During login, *bcrypt.compare()* was used to validate the user's input against the hashed version.

We also applied a reasonable salt round configuration to strike a balance between security and response time.

Input validation and sanitisation: Almost every route on the backend runs through input validation middleware using express-validator. This protects against:

- NoSQL injections
- Malformed input
- Unexpected data types
- XSS vectors in form fields

Model	Key fields
User	id, name, contact, role, status, permissions
Appointment	patientId, doctorId, date, status, timestamps
Group	name, members, adminPrivileges, category
Resource	title, fileUrl, fileType, author, uploadDate, accessControl

Table 2: Core collections used in the SereneCare backend